FORTUNE INNOVATIVES

# Phase 2 Implementation Report

## CRM & LMS — Complete Feature Documentation

| | |
|---|---|
| Report Date | **29 June 2026** |
| Repository Branch | **main** |
| Phase 2 FRs Completed | **11 / 11** |
| FRs Covered | **FR-08 → FR-18** |
| Supporting Sub-Features | **6** |
| Bug Fixes | **7+** |
| Status | **Shipped & deployed to production** |

# 1. Executive Summary

This report documents the full implementation of Phase 2 functional requirements (FR-08 through FR-18) and supporting bug fixes delivered for the Fortune Innovatives CRM & LMS. Every feature was built, type-checked, deployed to production via Render, and verified in the live system.

**Headline outcomes**

- **11 of 11 Phase 2 FRs delivered** — FR-08 (Student UPI Self-Pay), FR-09 (Collection Totals), FR-10 & FR-11 (Expenses & Utility Bills), FR-12 (CEO Check-In Notifications), FR-13 (Attendance Audit), FR-14 (Course-Wise Tracking), FR-15 (Equal Lead Distribution), FR-16 (Placement 2-Step Approval), FR-17 (Weekly Timetable), FR-17B (Slot Log + Missed-Hour Alerts), FR-18 (Student Feedback).
- **6 supporting sub-features** — Trainer placement page, on-the-fly company creation, student readiness snapshot, CEO/CP readiness pills, daily timetable history view, free-hour visibility for reviewers.
- **7+ bug fixes** — telecaller scope, trainer timetable scope, modal polish, late-logging UX, end-of-shift detection timing, free-hour visibility, dropdown clipping, plus earlier fixes (duplicate fee records, salary page performance, IST timezone consistency, CEO popup UTR display).
- **Additive-only schema changes** — multiple new tables, columns, and status values; no destructive migrations.
- **Production-ready** — backend auto-deployed via Render, frontend dist.zip rebuilt and current.

## 1.1 Phase 2 scope — all FRs delivered

| FR | Title | Page / Surface |
|---|---|---|
| FR-08 | Student UPI Self-Pay (manual verification) | Student Dashboard → Fees tab |
| FR-09 | Daily / Weekly / Monthly Collection Totals with drill-down | CEO Dashboard, /fees Collections tab |
| FR-10 & FR-11 | Expenses & Utility Bills module | /expenses |
| FR-12 | CEO notification on trainer/CP check-in & check-out | Notifications bell |
| FR-13 | Attendance Marking Audit Log | /attendance/audit |
| FR-14 | Course-Wise Tracking with KPI drill-downs | /courses → Insights modal |
| FR-15 | Equal Lead Distribution (round-robin to telecallers) | Backend logic + redistribution |
| FR-16 | Placement Readiness 2-Step Approval | /placement-readiness, /placements |
| FR-17 | Weekly Trainer/CP Timetable | /timetable |
| FR-17B | Daily Slot Log + Missed-Hour Alerts + free-hour logging + history view | /timetable + CEO Dashboard tile |
| FR-18 | Student Feedback admin page + low-rating alerts | /student-feedback |

# 2. Bug Fixes & Corrections

The following bugs were identified and resolved during the Phase 2 release cycle. Each is documented with symptom, root cause, and the fix that was deployed.

### Fix 1 — Telecaller dashboard showed entire branch leads

Commit 044e118 · Severity: High · Affected: Telecaller dashboards

**Symptom:** Both Telecaller 1 and Telecaller 2 (both Tiruppur) were each seeing identical 1,000-lead pipelines and identical "Total Assigned Leads" modal contents — defeating the round-robin distribution from FR-15.

**Root cause:** The `getLeads` endpoint was using `where.OR = [{ assignedToId }, { branchId }]` for telecaller roles, which leaked every branch lead to every telecaller in the branch.

**Fix:** Strict filter — `where.assignedToId = req.user.id` for the TELECALLER role. Each telecaller now sees only their own ~600 leads.

### Fix 2 — Trainer saw another trainer's row on co-taught batches

Commit 5dd775c · Severity: Medium · Affected: Trainer timetable view

**Symptom:** When trainer A was primary on a batch with trainer B as secondary, both rows rendered in trainer A's timetable. Trainers also couldn't see batches where they were the secondary.

**Fix:** `where` filter now uses `OR [trainerId, secondaryTrainerId]`; row builder only adds a row when the current user is the relevant trainer on that batch.

### Fix 3 — "LOG LATE" button missing on missed slots

Commit bb9f3c6 · Severity: Medium · Affected: Trainer logging UX

**Symptom:** Once a slot was auto-marked MISSED, trainers had no way to retroactively log what they actually taught.

**Fix:** Red "LOG LATE" button now appears on missed slots. Submit flips status back to LOGGED via existing upsert.

#### Fix 4 — Missed-hour alert fired too early

Commit b497e57 · Severity: High · Affected: All reviewer roles

**Symptom:** Red "missed teaching hour" alert fired the moment each 2-hour slot ended, even though trainers expected all day to log.

**Fix:** Constant `END_OF_SHIFT_MIN = 1200` (8 PM IST). Scheduler early-returns before that; after 8 PM, unlogged scheduled slots are marked MISSED once and CEO/ADMIN/CP notified.

#### Fix 5 — CEO/CP could not see free-hour logs

Commit 0533df6 · Severity: Medium

**Symptom:** Green "Free hour" tiles trainers logged were invisible to reviewers.

**Fix:** Backend now fetches free-hour logs for all trainers in scope and attaches them per-trainer as `freeHourBySlot`. CEO/CP see all (read-only); trainer remains the only one who can edit their own.

#### Fix 6 — Log modal had heavy dark backdrop

Commit 6ce4811 · Severity: Low (UX polish)

**Fix:** Dark overlay replaced with fully transparent backdrop. Modal still floats with a soft shadow and ring so it reads as elevated.

#### Fix 7 — Company & Student dropdowns clipped long lists

Commit f7f7951 · Severity: Low

**Fix:** `SelectContent` now has `max-h-72` with internal scrolling.

## 2.1 Earlier-cycle fixes worth noting

| Bug | Resolution |
| --- | --- |
| Duplicate fee records (e.g. Vinoth P appearing twice) | `getAllFeeRecords` now subtracts approved Fee Payment Request totals from admission.feePaid; only shows the admission row when remainder > ₹0.50. |
| Salary & Discipline page slow | Split into `loadStaticData` (mount only) + `loadTasksOnly` (filter change). Penalty checks moved to fire-and-forget. |
| Accountability modal slow on 1060+ items | `useMemo` filter + 100-item cap + "+more" footer. |
| Watchlist drill-down links 404'd | All 15 rule links rewired to existing list pages (e.g. `/telecaller/pipeline`, `/payroll?tab=X`). |
| Trainer/CEO attendance sync mismatch (IST timezone) | Created `backend/src/utils/istDate.ts` with `istMidnight`, `istDayRange`, `istTodayString`. All attendance read/write paths now use it. |
| CEO fee approval popup missing UTR (BF-07) | Added monospace UTR row + "Self-pay" amber chip to `FeeApprovalModal`. |
| CourseInsightsModal couldn't scroll | `DialogContent` uses `flex flex-col overflow-hidden`; inner uses `flex-1 overflow-y-auto`. |

# 3. Feature — FR-08 Student UPI Self-Pay

STUDENT (pay) | CEO (verify) | Page: Student Dashboard → Fees tab

Students can now pay their remaining fees directly from their dashboard by scanning the CEO's UPI QR code (auto-generated from the institute's UPI ID) and submitting the resulting Unique Transaction Reference (UTR) number for manual verification. The CEO approves or rejects each submission, and the student sees status updates in real time.

> **Manual verification — NOT Razorpay**
>
> Per the requirement, this is intentionally a manual UPI flow (CEO scans bank statement, matches the UTR, then approves in the CRM). No payment gateway fees, no PCI scope, no API integration burden. Just a managed verification workflow.

## 3.1 Student-facing flow

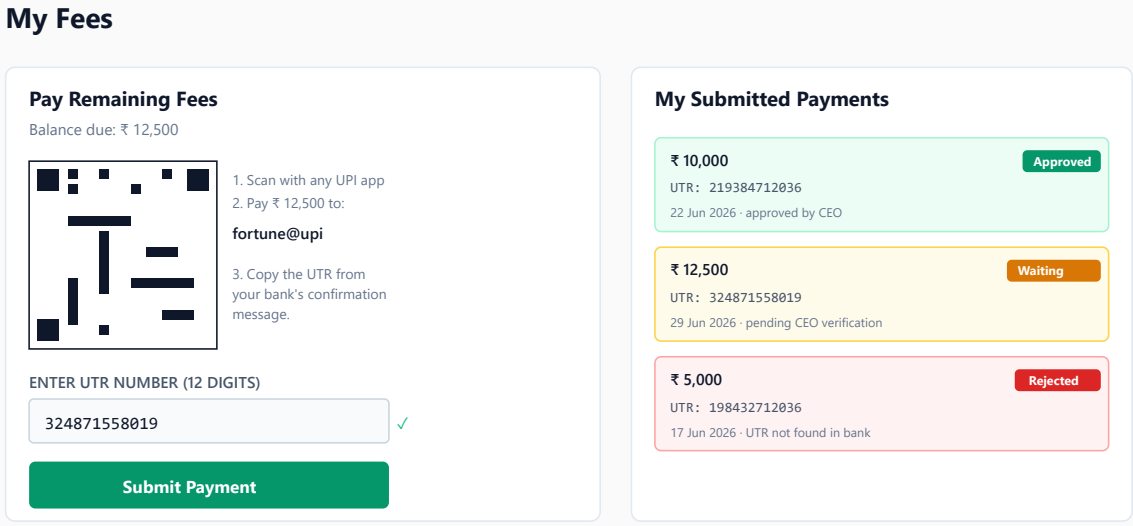



*Figure 3.1 — Student Dashboard Fees tab. Left: pay card with auto-generated UPI QR + 12-digit UTR input. Right: history of submitted payments with status banners.*

## 3.2 UTR validation rules

| Layer | Validation | Behavior on failure |
|---|---|---|
| Client (input) | Auto-strips non-digit characters; caps at 12 chars | Submit button disabled, inline error "Enter all 12 digits" |
| Server (regex) | `/^\d{12}$/` | HTTP 400 "Invalid UTR format" |
| Server (uniqueness) | UTR cannot match any existing APPROVED payment | HTTP 409 "This UTR is already approved for another payment" |

## 3.3 Step-by-step workflow

1. Student opens dashboard → Fees tab. Card "Pay Remaining Fees" shows the current balance from `admission.feeBalance`.
2. UPI QR is auto-generated using the institute UPI ID (from TermsSettings) via `api.qrserver.com`. Amount is pre-filled, so the student's UPI app shows the exact figure on scan.
3. Student scans QR with any UPI app (Google Pay, PhonePe, Paytm, BHIM) and completes payment from their phone.
4. Their bank sends a confirmation SMS with the UTR (a 12-digit number, e.g. `324871558019`).
5. Student types the UTR into the input on the dashboard. Live validation guides them to exactly 12 digits.
6. On Submit, the backend creates a `FeePaymentRequest` with status **PENDING** and attaches the UTR.
7. The card flips to "Waiting for approval" status. A new entry appears in "My Submitted Payments" with the orange Waiting badge.
8. The page auto-refreshes every 30 seconds and on window focus — so the student sees CEO action without manual reload.
9. CEO opens the Fees → Approvals page, sees the request with the UTR shown, matches it against the bank statement, and clicks Approve or Reject.
10. Approval → the badge flips to green "Approved", and admission.feeBalance is decremented. Rejection → red badge with the CEO's reason.

**EXAMPLE — Real conversation flow**

Anandhi V owes ₹ 12,500. She opens her phone, scans the QR on her student dashboard, transfers ₹ 12,500 to `fortune@upi` via Google Pay, and receives an SMS: "Sent ₹12,500 to FORTUNE INNOVATIVES. UTR: 324871558019." She types `324871558019` into the input field, submits. The CEO receives a notification, opens her bank statement on a laptop, confirms the UTR matches, clicks Approve. Anandhi's screen shows "Approved" within 30 seconds. Her balance drops to zero.

# 4. Feature — FR-09 Daily / Weekly / Monthly Collection Totals

CEO ADMIN Surfaces: CEO Dashboard + /fees Collections tab

The CEO Dashboard now shows three large clickable cards summarizing fee collections for Today, This Week, and This Month. Each card opens a drill-down modal listing every individual payment that contributed to the total. A dedicated Collections tab on the Fees page provides the same view with deeper filtering.

## 4.1 Dashboard quick-glance row

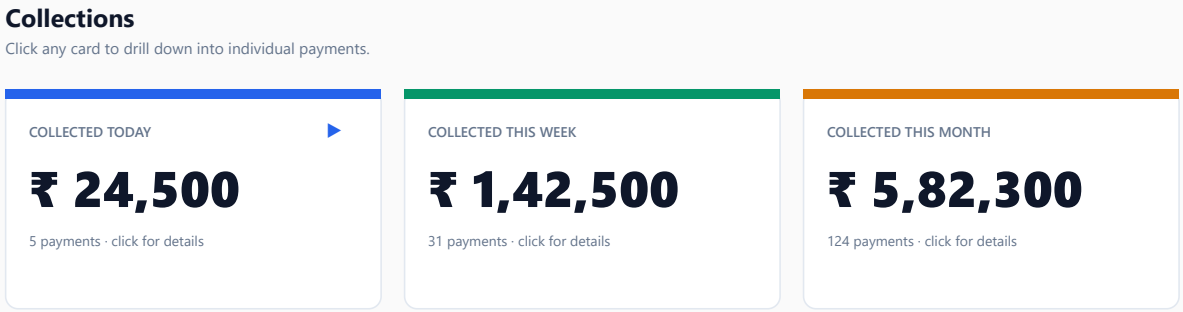



*Figure 4.1 — Collections row on the CEO Dashboard. Each card is fully clickable.*

## 4.2 Drill-down modal

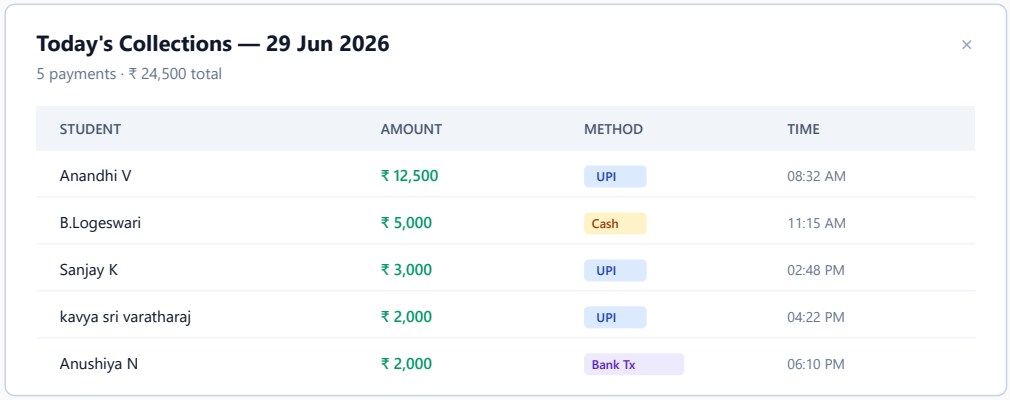



*Figure 4.2 — Drill-down modal opened by clicking "COLLECTED TODAY". Lists every payment with student, amount, method, and time.*

## 4.3 Fees → Collections tab

The Fees page has a dedicated **Collections** tab providing the same data with deeper analytics:

- **Daily / Weekly / Monthly toggle** — switch between time windows.
- **Bar chart** — visualize collection trends over the period.
- **Detailed table** — every payment with all metadata.
- **Method breakdown** — UPI / Cash / Bank Transfer / Cheque counts.

# 5. Feature — FR-10 & FR-11 Expenses & Utility Bills

CEO ADMIN CHANNEL PARTNER Page: /expenses

A new **Expenses** module was built to centralize tracking of all branch operational expenses including utility bills (FR-11). It supports 8 expense categories, recurring bills, monthly trend visualization, and full CRUD operations.

## 5.1 Eight expense categories

| Category | Examples |
|---|---|
| UTILITIES | Electricity bill, Water bill, Internet, Phone |
| SALARIES | Staff wages outside the payroll module |
| RENT | Branch rent, parking, equipment lease |
| SUPPLIES | Stationery, fabric, design materials, printing |
| MARKETING | Ads, flyers, social media spend, events |
| MAINTENANCE | Building repair, machine servicing, IT support |
| TRANSPORT | Fuel, taxi, courier, vehicle maintenance |
| OTHER | Anything that doesn't fit the above |

## 5.2 Page layout

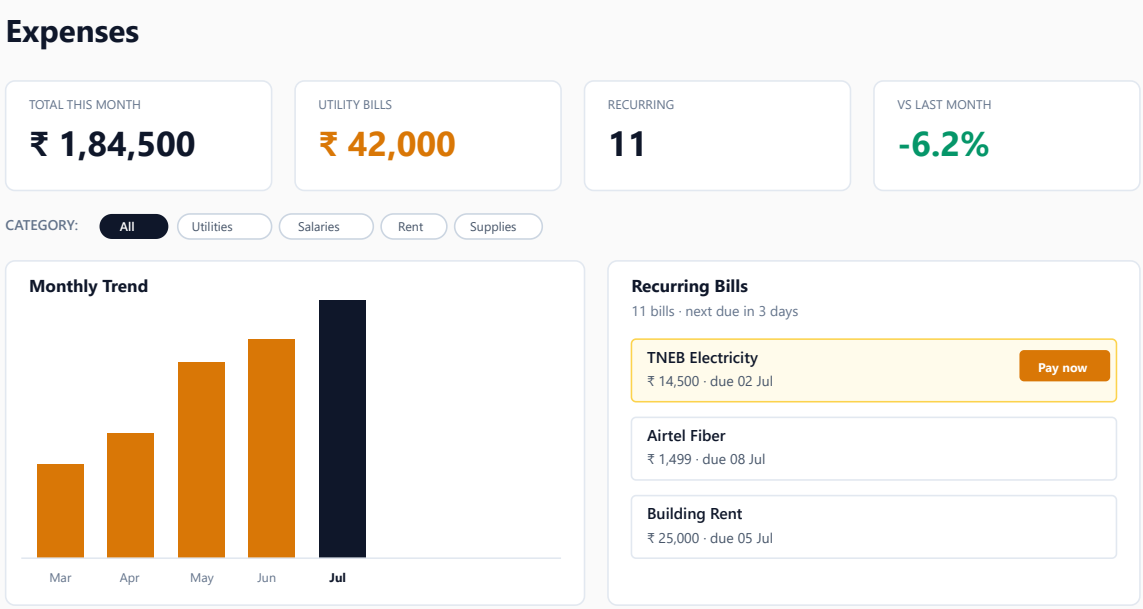



*Figure 5.1 — Expenses page: 4 KPI cards, category filter chips, monthly bar chart, and recurring-bills panel with payment due dates.*

## 5.3 Backend

- New `backend/src/modules/expenses/` module with `expense.controller.ts` and `expense.routes.ts`.
- **5 endpoints:** list, create, update, delete, and summary.
- Server-side validation of the 8-category enum on every write.
- Branch scoping: CHANNEL_PARTNER can only see and edit expenses in their own branch.

## 5.4 Recurring bills

An expense can be flagged `isRecurring=true`. When marked recurring, the entry serves as a template — the Recurring Bills panel surfaces upcoming bills with due dates, and the CEO can mark each one paid when settled. This is particularly useful for monthly utility bills (electricity, internet, water) that need to be tracked but where amounts vary.

# 6. Feature — FR-12 CEO Check-In & Check-Out Notifications

CEO (receives) ADMIN (receives) TRAINER (triggers) CHANNEL PARTNER (triggers)

The CEO and ADMIN roles now receive an automatic notification every time a trainer or Channel Partner self-checks-in or self-checks-out from their portal. This gives real-time visibility into staff presence without needing to monitor the attendance dashboard.

## 6.1 Triggers

| Action | Source endpoint | Notification recipients |
|---|---|---|
| Trainer self check-in | `POST /api/trainer-attendance/check-in` | CEO + ADMIN (org-wide) |
| Trainer self check-out | `POST /api/trainer-attendance/check-out` | CEO + ADMIN (org-wide) |
| CP / staff self check-in | `POST /api/staff-attendance/check-in` | CEO + ADMIN (org-wide) |
| CP / staff self check-out | `POST /api/staff-attendance/check-out` | CEO + ADMIN (org-wide) |

## 6.2 Notification format

**Example notifications in the bell**

- **"Keerthi Fortune checked IN"** — Trainer at Tiruppur · 08:32 AM IST
- **"AJAY FORTUNE checked OUT"** — Trainer at Tiruppur · 06:15 PM IST
- **"Vasu Devan checked IN"** — Channel Partner at Tiruppur · 09:05 AM IST

## 6.3 Implementation notes

- Notifications are sent via the existing `NotificationService`: `notifyRole('CEO', ...)` and `notifyRole('ADMIN', ...)`.
- All calls are **fire-and-forget** — a notification failure cannot prevent the check-in/out from succeeding.
- The trainer's full name is included in the notification — the lookup query was extended to include `user: { select: { firstName: true, lastName: true } }`.
- Each notification links to `/attendance` so a click takes the reviewer straight to the live attendance widget.

# 7. Feature — FR-13 Attendance Audit Log

CEO ADMIN CHANNEL PARTNER Page: /attendance/audit Commit: b8e2175

Every attendance write across students, trainers, and staff is now permanently recorded with a full audit trail. The system captures who pressed the button, when, what was set, what changed (before → after), the method (manual, QR, GPS, self check-in, or leave sync), the source endpoint, IP address, and user agent of the requester.

## 7.1 Use cases

**EXAMPLE — Dispute resolution**

A student claims they were present on 25 June but were marked absent. The CEO filters by Kind=Student, Date=25 Jun, searches the student's name. The result shows exactly who marked the student absent, from which device, via which method, at what time. Informed correction follows.

**EXAMPLE — Detect back-dating**

A trainer tries to retroactively mark Monday's attendance at Friday 11 PM. The audit log captures the action timestamp (Friday 11 PM) and the attendance date (Monday) — the mismatch is permanently visible.

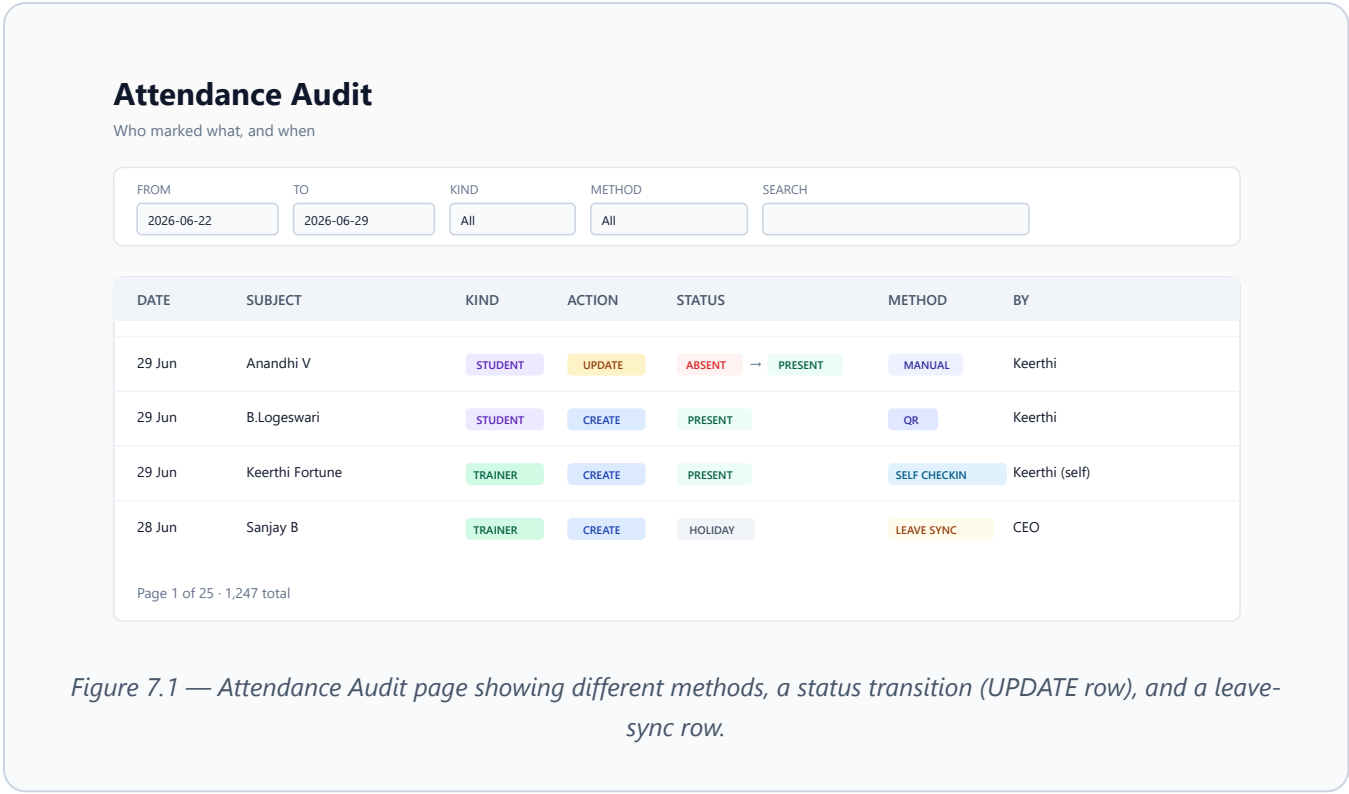



*Figure 7.1 — Attendance Audit page showing different methods, a status transition (UPDATE row), and a leave-sync row.*

## 7.2 Hook coverage — 13 write sites

- `attendance.controller.ts` — student create + student update (manual)
- `trainer.attendance.controller.ts` — trainer manual mark + trainer self check-in + trainer self check-out
- `staff-attendance.controller.ts` — staff/CP self check-in + staff/CP self check-out
- `qr.controller.ts` — QR entry attendance + QR exit update
- `leaves.controller.ts` — leave-approval syncs to trainer attendance (create + update) and to staff attendance (create + update)

# 8. Feature — FR-14 Course-Wise Tracking

CEO ADMIN CP (own branch) TRAINER (own courses) Page: /courses → Insights modal

Each course on the Courses page now exposes a "View Insights" button that opens a modal with 4 KPI cards summarizing the course's performance. Each KPI card is itself clickable — clicking opens a stacked drill-down dialog listing the underlying students, admissions, revenue line items, or attendance entries that make up the number.

## 8.1 Four KPIs per course

| KPI | What it counts | Drill-down content |
|---|---|---|
| Students Enrolled | Total active students currently in this course | List of students with name, batch, branch, enrollment date |
| Admissions This Month | New admissions in the current month | List of new admissions with date, fee paid, source |
| Total Revenue | Sum of all approved payments for this course | List of payment line items (student, amount, method, date) |
| Attendance Rate | Avg attendance % across all batches teaching this course | Per-student attendance breakdown with batch context |




*Figure 8.1 — Course Insights modal: 4 clickable KPI cards across the top, enrollment trend chart below.*

## 8.2 Backend

- New `backend/src/modules/courses/course.insights.controller.ts` (~470 LOC + breakdown handlers).
- **Three endpoints:**
  - `GET /api/courses/:id/insights` — KPI summary for a single course
  - `GET /api/courses/summary` — KPI summary for all courses (for the Courses page card strip)
  - `GET /api/courses/:id/insights/breakdown?kind=...` — drill-down for a specific KPI (students, admissions, revenue, attendance)
- 17 Prisma queries per insights call, all wrapped in safety try/catch so a partial failure still returns the parts that worked.
- Branch scoping: CHANNEL_PARTNER sees only their own branch. TRAINER must have a `CourseTrainer` link to access the course. STUDENT and TELECALLER receive HTTP 403.

## 8.3 Stacked drill-down dialog

Clicking any KPI card opens a second dialog stacked on top of the modal, listing the underlying records. The original Insights modal stays visible behind the drill-down so the context (which course) is never lost.

# 9. Feature — FR-15 Equal Lead Distribution

TELECALLER (receives leads) Backend: createPublicLead

Public-form leads (submitted by prospective students via the website) now auto-distribute to the available telecallers using a round-robin **least-loaded** strategy. The previous behavior dumped all public leads onto the CEO's queue, which then required manual reassignment.

## 9.1 Distribution algorithm

1. When a public lead arrives via `POST /api/leads/public`, the backend runs a `groupBy` query counting active leads per telecaller.
2. The telecaller with the fewest currently-assigned active leads wins (lowest count). In case of a tie, the first one returned by the query is picked.
3. The new lead's `assignedToId` is set to that telecaller's user ID.
4. The telecaller receives a notification: "New lead assigned: [Name]" with a click-through to the lead detail.

> **Historical redistribution**
>
> A one-off migration was run during the rollout to redistribute 188 existing leads that had been incorrectly assigned to the CEO. The script re-allocated them by round-robin between Telecaller 1 and Telecaller 2, resulting in TC1=605 and TC2=604 active leads (a difference of just one, which is the theoretical optimum for an odd total of 1,209).

## 9.2 Why this matters

| Before | After |
|---|---|
| All ~1,000 public leads landed in CEO's queue. CEO manually reassigned each one or telecallers had to fish for unclaimed leads. | Public leads instantly land in whichever telecaller's queue is currently smallest. CEO's queue stays at zero. Telecallers see new leads as soon as they're submitted. |
| No accountability — if a lead sat unworked, the CEO got the watchlist warning even though no telecaller had been assigned. | Accountability is automatic — every public lead has a real owner from the moment it's created, and the Accountability Watchlist correctly flags the assigned telecaller if it goes stale. |

# 10. Feature — FR-16 Placement Readiness 2-Step Approval

TRAINER (submit) CEO (approve) ADMIN (approve) CP (own branch)

Pages: /placement-readiness · /placements

Placement submissions from trainers no longer land directly in the placement queue. They sit in a "Pending Approvals" tab where CEO/ADMIN/CP review, approve, or reject with a written reason. The branch placement counter is only incremented after approval. Five sub-features support the core flow.

## 10.1 Trainer-facing submission page

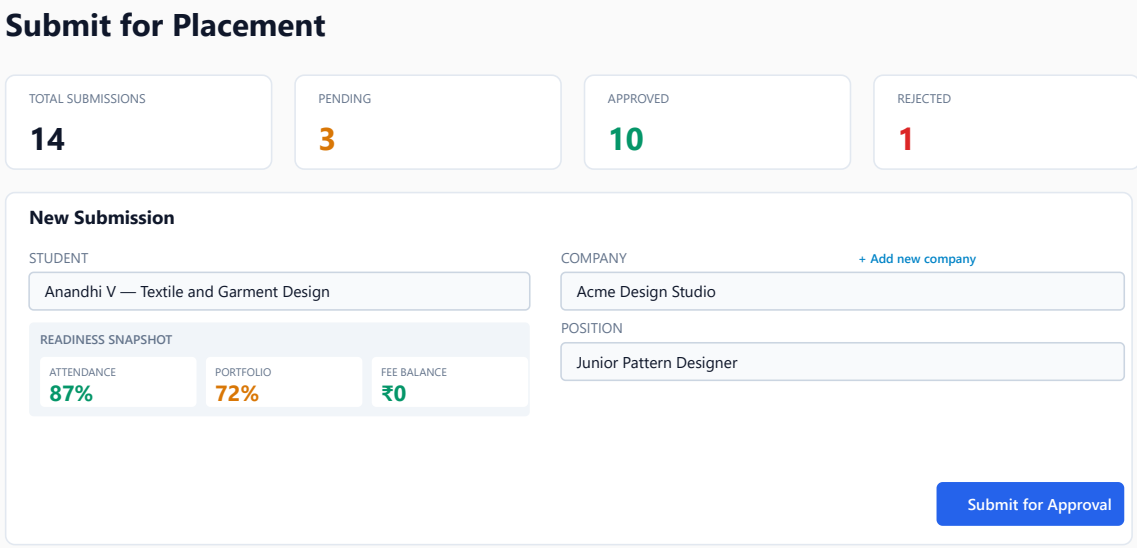



*Figure 10.1 — Trainer "Submit for Placement" page with KPI strip, Readiness Snapshot, and on-the-fly company creation.*

## 10.2 CEO Pending Approvals queue with readiness pills

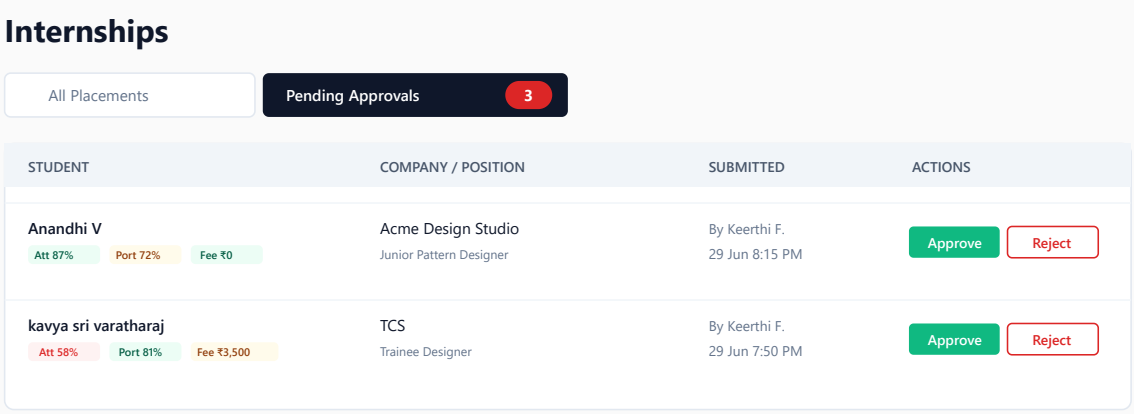



*Figure 10.2 — CEO Pending Approvals with readiness pills inline. Contrast Anandhi (mostly green, easy approve) vs kavya (red attendance + outstanding fee — needs scrutiny).*

## 10.3 Approve / Reject flow

**EXAMPLE — Approve**

1. CEO clicks the green Approve button.
2. Backend: `status = ELIGIBLE`, `approvedById` + `approvedAt` set, branch `totalPlacements` atomically incremented.
3. Trainer receives a notification "Placement readiness approved" with a link to /placements.
4. Row disappears from Pending; appears in the regular All Placements view.

**EXAMPLE — Reject**

1. CEO clicks red Reject — modal opens with required reason textarea (min 3 chars).
2. Backend: `status = REJECTED`, `rejectedAt` + `rejectionReason` set (capped at 500 chars). Counter NOT incremented.
3. Trainer receives a notification with the rejection reason snippet.
4. Trainer's "My Submissions" table shows the row as REJECTED with the reason visible on hover.

**Idempotency & state guards**

Both endpoints verify `status === PENDING_READINESS` before flipping. An already-approved or already-rejected record cannot be re-processed (returns HTTP 400). Branch counter is guaranteed to increment exactly once.

# 11. Feature — FR-17 Weekly Trainer/CP Timetable

CEO ADMIN CP (own branch) TRAINER (own row) Page: /timetable

A single-page weekly view showing every active batch, grouped by trainer, across six 2-hour time slots from 8 AM to 8 PM. Drawn directly from existing Batch records — no separate scheduling system needed.

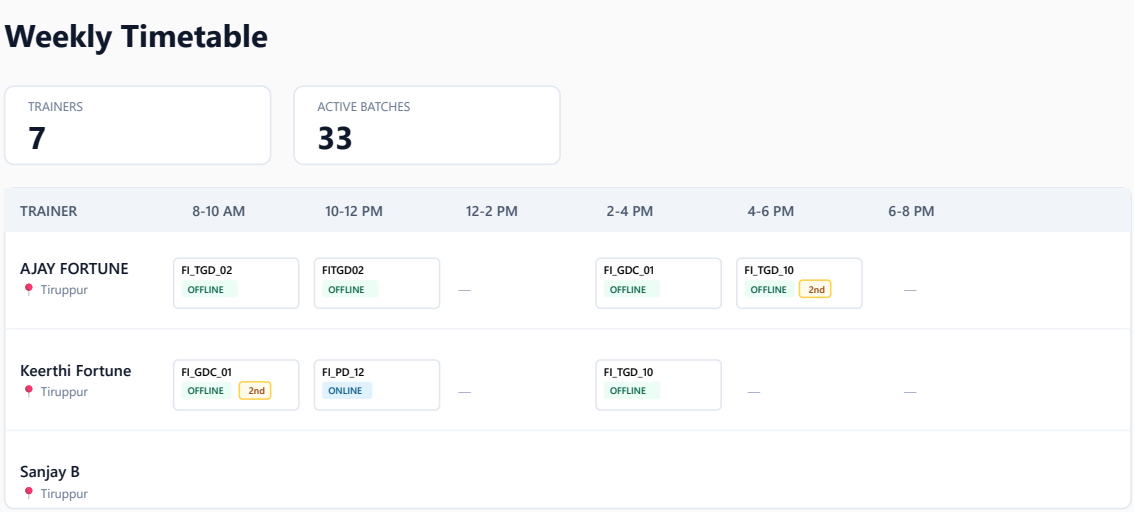



*Figure 11.1 — Weekly Timetable grid: trainers row, six time slots, chips with course code + mode badge + "2nd" badge for secondary-trainer batches.*

## 11.1 Six time slots (8 AM - 8 PM)

| Slot ID | Time range (IST) |
|---|---|
| `slot-08` | 8:00 – 10:00 AM |
| `slot-10` | 10:00 AM – 12:00 PM |
| `slot-12` | 12:00 – 2:00 PM |
| `slot-14` | 2:00 – 4:00 PM |
| `slot-16` | 4:00 – 6:00 PM |
| `slot-18` | 6:00 – 8:00 PM |

## 11.2 Role-based scoping

| Role | Sees |
|---|---|
| CEO / ADMIN | All trainers across all branches |
| Channel Partner | Only trainers in their own branch (server-forced) |
| Trainer | Only their own row, including secondary-trainer batches |

# 12. Feature — FR-17B Slot Logging & Missed-Hour Alerts

TRAINER (log) CEO (review) ADMIN (review) CP (review)

Trainers log what they taught in each 2-hour slot through the day. Empty slots get a "free hour" log option for non-batch work. At end of shift (8 PM IST), any unlogged scheduled slot triggers a red alert to CEO, ADMIN, and branch CP.

## 12.1 The four slot states

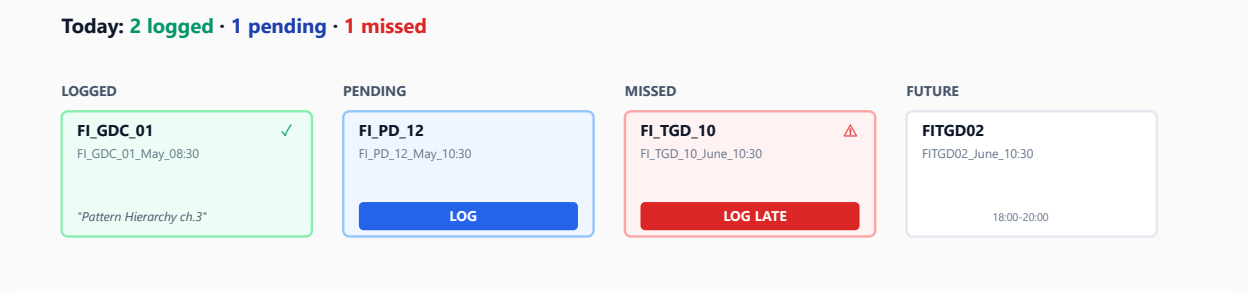



*Figure 12.1 — Four slot states. LOGGED (green ✓), PENDING (blue + LOG button), MISSED (red + LOG LATE), FUTURE (neutral).*

## 12.2 End-of-shift detection rule

**The 8 PM IST rule**

Past slots stay in PENDING state throughout the working day. The scheduler runs every 5 minutes but does NO work before 8 PM IST. This gives trainers the entire day to catch up. Only after 8 PM does the scheduler scan for unlogged scheduled slots and mark them MISSED, fire notifications, and surface red alerts.

## 12.3 Free-hour logging

Empty cells in the trainer's row show a dashed **"+ Log free hour"** button. Trainers click it to record administrative, preparation, or other non-batch work. The cell flips to a green "Free hour" tile. CEO, ADMIN, and CP see these tiles read-only for all trainers in scope.

**EXAMPLE — A typical Tuesday for Keerthi**

FI_GDC_01 (8:30-10:30): logged "Pattern Hierarchy ch.3". FI_PD_12 online (10:30-12:30): logged. 12-2 PM free: logged "Reviewed portfolio submissions". FI_TGD_10 (2-4 PM): logged. 4-6 PM free: logged "Built slide deck for tomorrow". 6-8 PM: empty, no work. At end of day, the CEO can see exactly how Keerthi used every hour of the day.

## 12.4 Three coordinated red surfaces for reviewers

- **CEO Dashboard tile** — "Missed Teaching Hours Today" with a red counter, clickable to /timetable
- **Timetable page banner** — red callout at the top with expandable per-trainer drill-down
- **Red chips in the grid** — each missed slot is itself visually marked

## 12.5 Daily history view

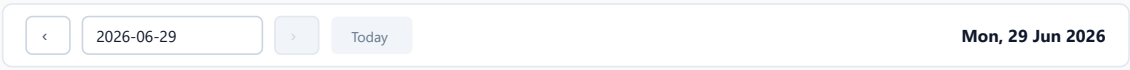



*Figure 12.2 — Date navigator strip. All four roles can browse past days; future days disabled at the picker.*

All four roles (CEO, ADMIN, CP, Trainer) can scroll back to past days to see historic logs and missed entries. Past days are strictly read-only — log buttons hidden, free-hour tiles non-clickable. Auto-refresh is disabled for past days.

# 13. Feature — FR-18 Student Feedback

CEO ADMIN CHANNEL PARTNER Page: /student-feedback

Surfaces the existing trainer-rating data into a dedicated CEO/ADMIN/CP review page with KPI cards, filters, trainer breakdown, and individual feedback rows. Low-rating alert fires automatically when any student rates a trainer 1 or 2 stars.

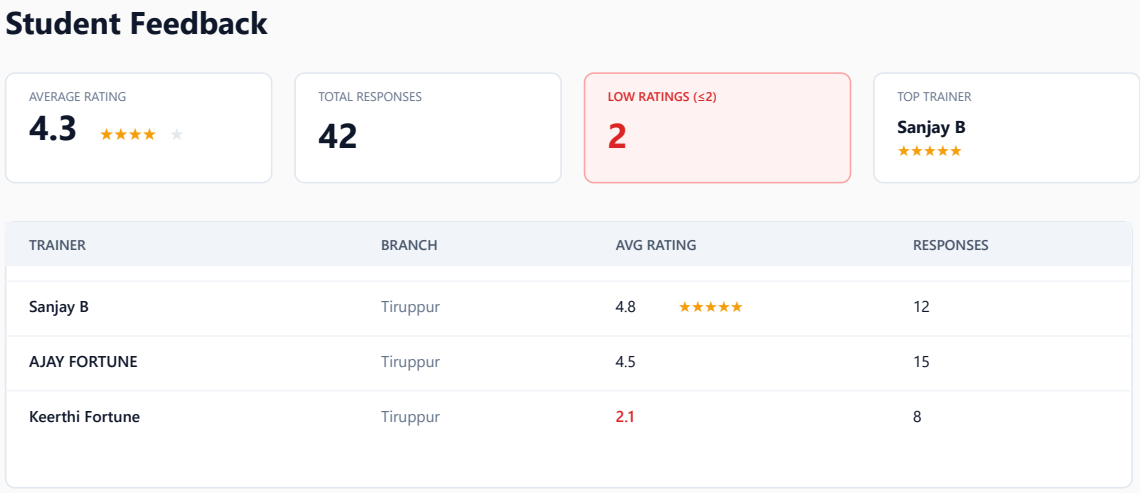



*Figure 13.1 — Student Feedback admin page: 4 KPI cards (red accent on Low Ratings), trainer breakdown table sorted by average rating.*

## 13.1 Low-rating alert

1. Student rates trainer 1 or 2 stars from their dashboard.
2. Backend's `submitTrainerRating` detects rating ≤ 2 and triggers a fire-and-forget IIFE.
3. Notifications sent to: CEO globally, ADMIN globally, the trainer's branch CP.
4. Title: "Low rating alert (X/5)". Message includes student name, trainer name, branch, and comment snippet (first 120 chars).
5. Link routes to `/student-feedback`.